Tema 5 Aprendizaje Automático

Autor: Ismael Sagredo Olivenza

Colaboración en este material de Belén Díaz Agudo y otros profesores de ISIA

Tema 5.1 Introducción al Aprendizaje Automático

En la semana pasada

- Algoritmos genéticos
- Estrategias Evolutivas
- Sistemas emergentes
- Otros paradigmas

Esta semana

- Introducción al Machine Learning
- Tipos de algoritmos de machine learning
- Tipos de problemas de machine learning
- Técnicas de Aprendizaje Automático clásico

5.1.1 Introducción al Aprendizaje Automático (Machine Learning)

Aprendizaje

- “El acto, proceso o experiencia de adquirir conocimiento o aptitudes”

Automático

- Dicho de un mecanismo o aparato que funciona en todo o en parte por sí solo.

5.1.1.2 Definición de Machine learning

Arthur Samuel (1959):
"Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed"

Tom Mitchell (1998)

"Well-posed Learning Problem: A computer program is said to learn from experience E with respect to some **task** (T) and some **performance** measure (P), if its performance on T, as measured by P, improves with **experience** E"

Machine Learning (ML) visto como un agente

El sistema es capaz de aprender a resolver un problema, esto sitúa a los agentes basados en machine learning **dentro de los agentes que aprenden** y **se adaptan a entornos dinámicos**.

ML permiten **predecir el comportamiento de un sistema a futuro**. Desde este punto de vista, podemos englobarlos como agentes deliberativos o basados en objetivos aunque podría construirse un agente puramente reactivo con ellos. Como sucedía con algoritmos genéticos, no son capaces de generar un plan si no de predecir un situación futura (salvo que entrenes al modelo para generar planes) y aunque haya un plan implicito, no suele ser verificable.

¿Cómo aprenden las máquinas?

No está muy claro cuál es el mecanismo interno del aprendizaje del ser humano (o de otros animales)

Aprendizaje: la capacidad que tienen los seres vivos de mejorar sus aptitudes.

Se requiere **memoria** para almacenar el nuevo conocimiento.

Se requiere **abstraer el problema** para permitir enfrentarse a problemas nuevos similares.

La mayoría de las técnicas de Machine learning realiza un **aprendizaje inductivo**, es decir, extrae conclusiones en base a ejemplos. Para ello, el algoritmo debe construir un modelo en base a los ejemplos que se le proporciona, o lo que es lo mismo, busca un patrón en los datos de entrada. Para poder hacer esta tarea, se requiere **abstraerse** de los pequeños detalles que no aportan información relevante para los casos generales.

Al hacer esta abstracción se introducen **sesgos** (Bias en inglés). Aunque parezca contradictorio, **es esencial esta inclusión de sesgos** en los datos, para que los modelos de machine learning sean aplicables a ejemplos no usados en el entrenamiento. A esto se le conoce como tener **capacidad de generalización**

Es decir, para poder generalizar, el modelo debe asumir un cierto error. Este debe ser bajo, pero normalmente (salvo que tengamos todo el conocimiento del dominio en los datos aprendidos) no será 0. Un error de 0 implicaría que el modelo se ha adaptado totalmente a los datos de entrenamiento, pero esto peude ser contraproducente si queremos tener capacidad de predecir escenarios no vistos en el entrenamiento.

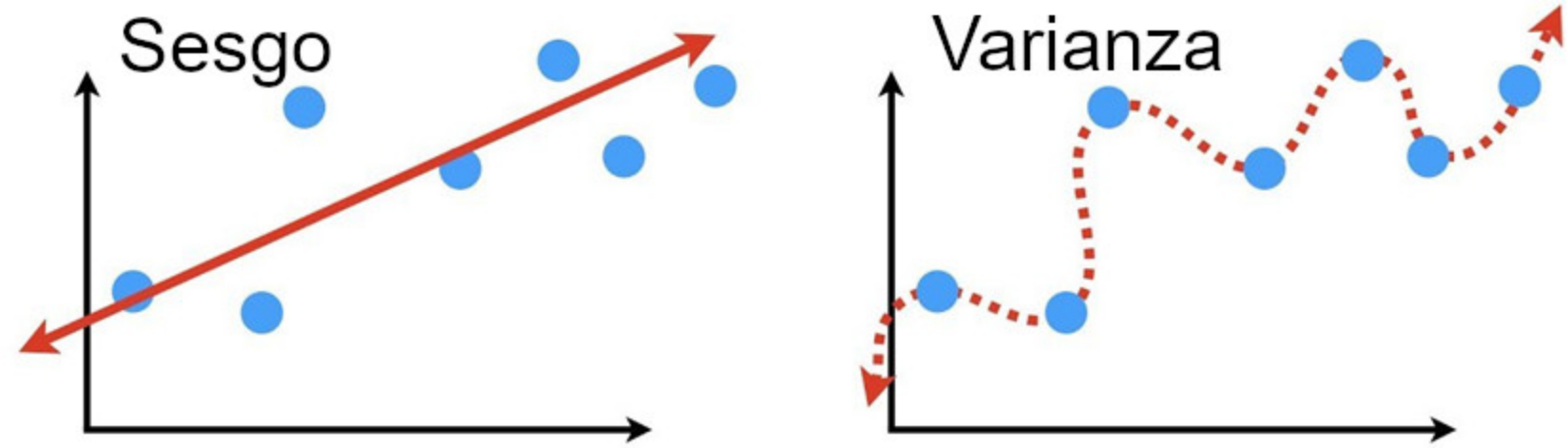
¿Los humanos introducimos sesgos en nuestro aprendizaje?

Constantemente:

- Reglas ortográficas (que algunas tienen excepciones)
- Las ecuaciones de carga eléctrica asumen que los cuerpos están en el vacío
- El movimiento uniformemente acelerado asume que lo haces en el vacío (sin fricción) y que la tierra no es curva (ni el espacio es curvo por la fuerza de la gravedad)
- La famosa ecuación de la relatividad de Einstein (e=mc2), en realidad solo se aplica en los casos donde haya partículas con masa (no aplicable a la luz por ejemplo)

Sesgo: es capaz de construir un modelo simplificado del problema, intentando extraer las características generales que los describen. El sesgo introduce un error, pero dicho error nos ayuda a predecir el comportamiento.

Varianza: no introduce sesgos y esto es algo que normalmente no queremos. También se le denomina **sobre adaptación** (overfiting)



Fuente https://skillsfuturetv.com/wp-content/uploads/2020/05/maxresdefault-360.jpg

Tipos de aprendizaje automático

- **Aprendizaje Supervisado**: requieren que los ejemplos que se introducen para aprender tengan también la solución al problema.
- **Aprendizaje No supervisado**: se lleva a cabo sobre ejemplos que no informan del resultado esperado
- **Aprendizaje por Refuerzo**: la retroalimentación la obtienen del mundo exterior (del entorno en el caso de un agente). Por tanto aprenden a base de prueba y error.

Tipos de problemas que se pueden abordar

- **De clasificación**: identificar a cuál de un conjunto de categorías pertenece una nueva observación. Se espera una salida discreta
- **De regresión**: La salida que se espera es continua. Un número de casos infinito => estimar la salida con la mayor precisión posible
- **De agrupamiento**: Si no se conoce a priori el número de clases ni el tipo de estas y lo que se pretende es precisamente conocerlas
- **De reducción de dimensiones**: Extraer características revelantes
- **De asociación**: Establece relaciones entre variables
- **De generación de contenido**: Genera nuevo contenido basado en los datos con los que se ha entrenados previamente.

5.1.2 Aprendizaje Supervisado

Intenta deducir una función que permita obtener a partir de unos datos de entrada la solución deseada. Tanto los datos de entrada como el valor deseado se proporciona al sistema. Al conocer la solución, el algoritmos puede estimar como de lejos está de dicha solución usando el error. Cada instancia de entrenamiento debe presentarse como un vector de entrada y una salida esperada.

Peso	Altura	Edad	Años Estudio	=	Salario en miles
20	170	30	10		**30**
27	175	35	5		**25**

5.1.2.1 Representación de los datos

Los datos se representan en forma de matriz de **nxm** siendo n el número de instancias que disponemos para entrenar el algoritmo y m el número de atributos que disponemos para cada ejemplo.

Las variables pueden ser de carácter cuantitativo o categóricas (Cualitativas).

Las instancias de entrenamiento **son puntos en el espacio m dimensional**. No podemos visualizar más de 3 dimensiones pero podemos visualizarlas de 2 en 2 o visualizar las más relevantes (más adelante veremos métodos de como hacerlo)

5.1.2.2 Una correcta representación

- Los individuos deben ser correctamente codificados, **evitar** en lo posible **el ruido** (datos con errores o con atributos que no aporten nada). Esto implica limpiar los datos, analizarlos, transformarlos…
- Debe haber **suficientes ejemplos** como para cubrir una buena parte del espacio del problema que se pretende resolver.
- Los datos a ser posible deben estar en la misma escala (0,1) o (-1,1).
- Discretización de variables numéricas
- Convertir categorías en valores binarios (**one-hot-encoding**)

	Nacionalidad
id1	Español
id2	Frances

	Español	Francés
id1	1	0
id2	0	1

5.1.2.3 One-hot encoding

Al trabajar con variables categóricas, una de las cuestiones más importantes es transformar nuestras variables categóricas en variables numéricas.

One-Hot Encoding transforma una característica categórica en tantas variables como valores posibles tenga la categoría y asigna a cada una de ellas 0 ó 1 en función del valor.

```
from sklearn.preprocessing import OneHotEncoder
oneHotEncoder = OneHotEncoder(drop='first')
oneHotEncoderFit = oneHotEncoder.fit(provincias.reshape(-1,1))
oneHotEncoderFit.transform(provincias.reshape(-1,1)).toarray()
```

5.1.2.4 Preparación de los datos

- Tratamiento de valores perdidos o nulos
  - Se pueden descartar las filas sin son pocos.
  - Calcular el punto medio en variables numéricas
  - Seleccionar el valor modal en las cualitativas

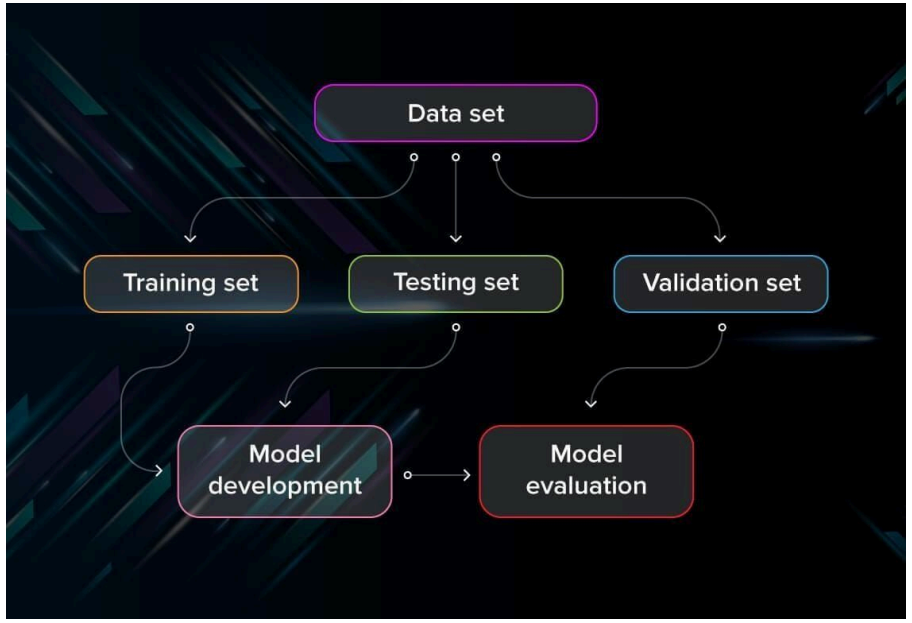
```
from sklearn.impute import SimpleImputer
```

- Seleccionar los más parecidos e intentar inferir cual sería el valor de las variables que faltan con algún tipo de interpolación...

```
from sklearn.impute import KNNImputer
```

5.1.2.5 Capacidad de generalización

Necesitamos que el sistema prediga la salida de instancias no usadas en el entrenamiento. Debemos evaluar al algoritmo no solo por el error que cometer con los datos de entrenamiento si no también con datos no usados en el entrenamiento.



5.1.2.6 Testing

Dividir el conjunto de datos en datos de entrenamiento y datos de prueba.

La distribución de los datos entre prueba y entrenamiento debe ser correcta y proporcional al número de clases. Una distribución sesgada (Biases) puede causar problemas.

SKLearn nos puede ayudar en esta tarea:

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(data, y, train_size = 0.8, random_state = 1234)
```

Durante la fase de entrenamiento, no sólo debemos fijarnos en el error de entrenamiento, sino también en el **error de validación**. El error de validación sería similar al **error de prueba**, pero se ejecuta durante el entrenamiento para ver si el modelo está perdiendo capacidad de generalización.

A priori podría parecer que la validación y la prueba son lo mismo, pero estos dos conjuntos se suelen diferenciar porque cuando se prueban diferentes ajustes del modelo (hiperparámetros) existe **aún un riesgo de sobreajuste** en el **conjunto de prueba** porque los parámetros se pueden **ajustar hasta que el estimador funcione de forma óptima**. De este modo, el conocimiento sobre el conjunto de pruebas puede "filtrarse" en el modelo y las métricas de evaluación ya no informan sobre el rendimiento de la capacidad de generalización.

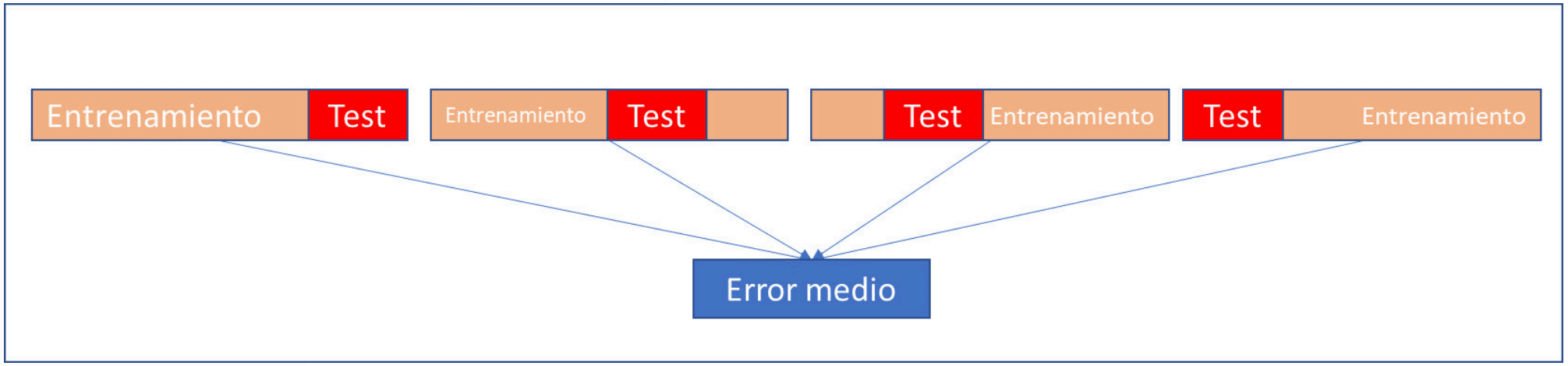
Por lo tanto, tendremos los siguientes conjuntos:

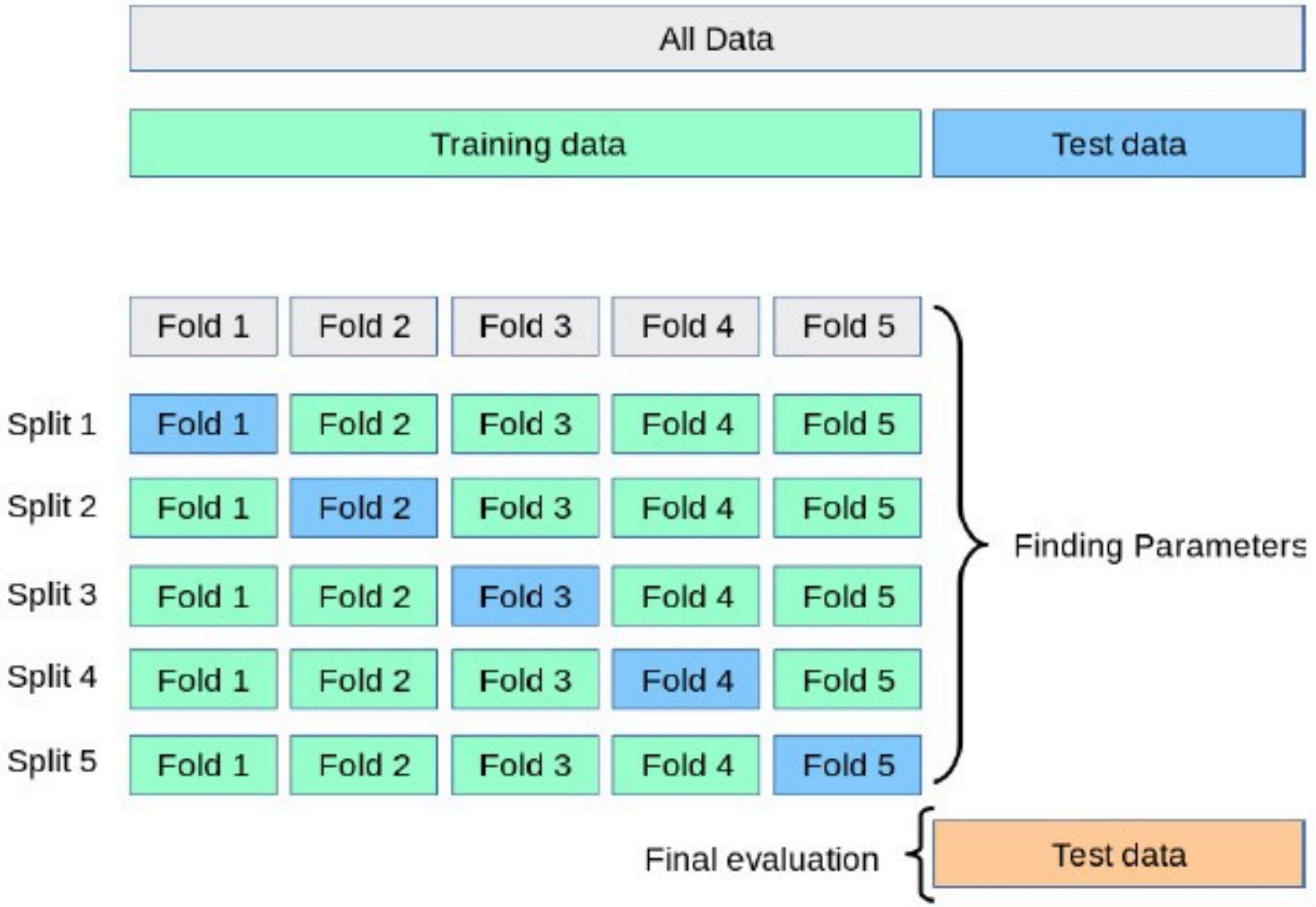
- **Error de entrenamiento**: Error de datos de entrenamiento.
- **Error de validación** (validation): Error de validación durante el entrenamiento para mantener la capacidad de generalización.
- **Error de pruebas** (test): Error con datos completamente excluidos del entrenamiento que tiene lugar después del entrenamiento.

Sin embargo, al dividir los datos disponibles en tres conjuntos, reducimos drásticamente el número de muestras que pueden utilizarse para el aprendizaje del modelo, y los resultados pueden depender de una elección aleatoria concreta para el par de conjuntos (entrenamiento, validación). Una solución es la **validación cruzada** o **cross-validation**

5.1.2.7 Cross Validation (K-Fold)

Cross validation o la validación cruzada permite que el porcentaje de los datos de entrenamiento no sea siempre el mismo y se calcula el error medio de cada división realizada (por ejemplo 4 si los datos de entrenamiento son el 25%). Esto nos permite aprovechar los datos de entrenamiento y evitar posibles sesgos en la elección del conjunto.




All Data
Training data
Test data
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Split 1
Split 2
Split 3
Split 4
Split 5
Finding Parameters
Final evaluation
Test data

5.1.2.8 ¿Y cómo medimos el error?

- **Error cuadrático medio** (Mean Square Error) entre el pronostico y el real. También el error absoluto medio

$$MSE(Y, Y') = \frac{1}{N} \cdot \sum_{i=1}^{N} (y_i - y'_i)^2$$

- **Matriz de confusión** (Ejemplo para clase binaria)

	C observada T	**C observada F**
C pronosticada T	True Positive (TP)	False Positive (FP)
C pronosticada F	False Negative (FN)	True Negative (TN)

- **Accuracity** o tasa de exactitud: Aciertos entre el total.

$$Exactitud = \frac{TP + TN}{TP + TN + FP + FN} = \frac{V}{N}$$

La tasa de aciertos puede presentar problemas en casos en los que las clases estén desequilibradas

- Tasa de verdaderos positivos (**Recall**). ¿Qué proporción de positivos reales se identificó en forma correcta?

$$Recall = \frac{TP}{P} = \frac{TP}{TP + FN}$$

- Valor Predictivo Positivo (**Precision**). ¿Qué proporción de identificaciones positivas fue correcta?

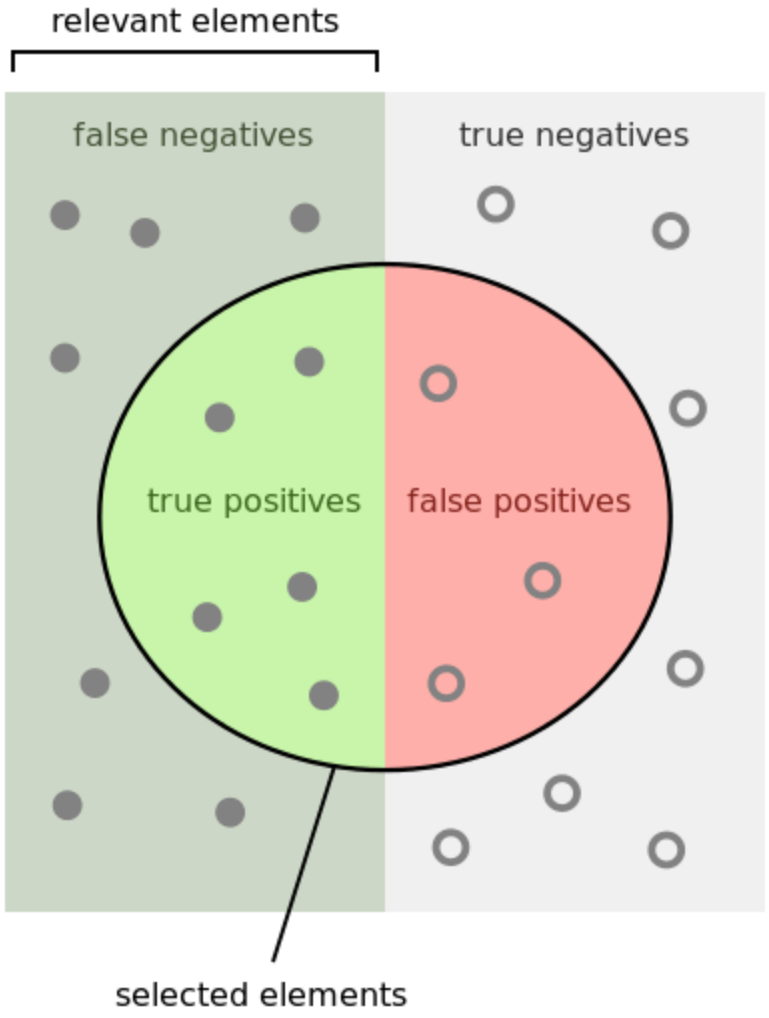
$$Precision = \frac{TP}{TP + FP}$$

- **F1 Score** $F1Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$

		Actual Outcome: Positive	Actual Outcome: Negative	
Predicted Outcome	Positive	True Positive	False Positive	Positive Predicted Value = TP / (TP + FP)
	Negative	False Negative	True Negative	

		Actual Outcome: Positive	Actual Outcome: Negative
Predicted Outcome	Positive	True Positive	False Positive
	Negative	False Negative	True Negative
		Sensitivity = TP / (TP+FN)	

5.1.2.8 Precision-Recall explicación gráfica



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{selected elements}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{relevant elements}}$$

Ejemplo

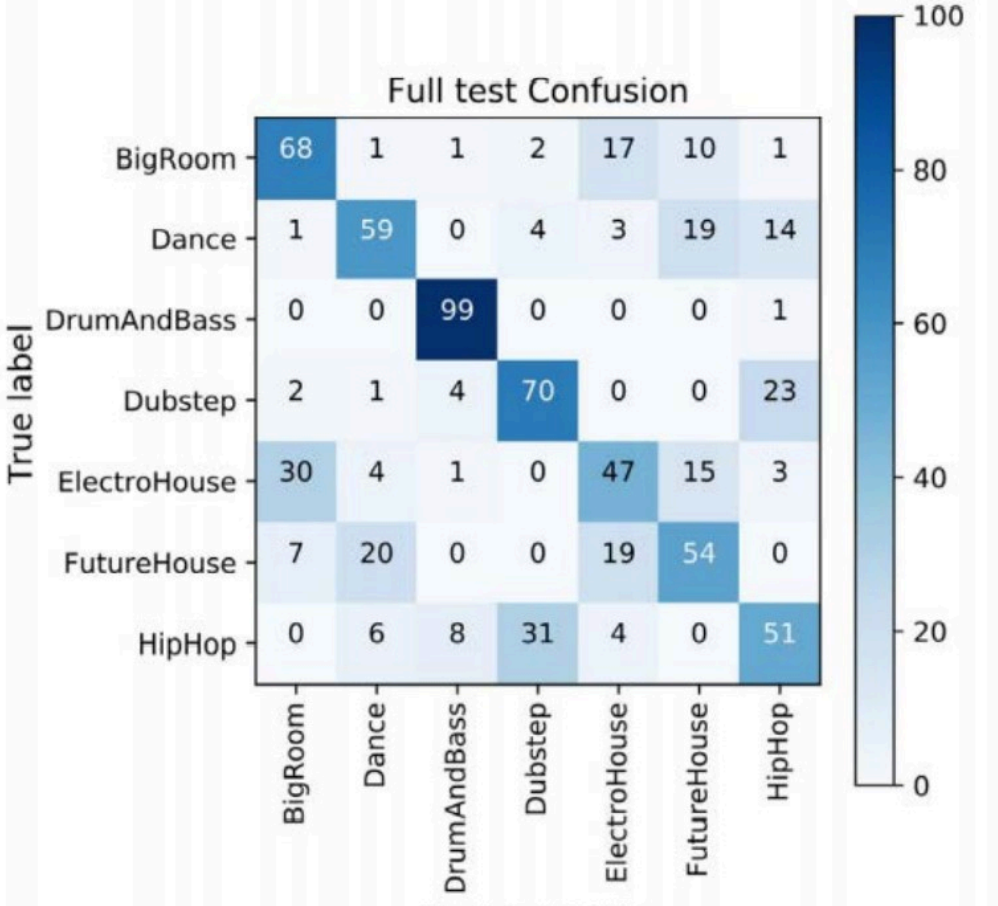
Dada la matriz de confusión: Calcula precision, recall, F1Score.

	C observada T	C observada F
C pronosticada T	176 (VP)	107 (FP)
C pronosticada F	316 (FN)	993 (VN)

- Precission: $= \frac{TP}{TP+FP} = \frac{176}{176+107} = 0,6219 = 62,19\%$
- Recall: $= \frac{TP}{TP+FN} = \frac{176}{176+316} = 0,3577 = 35,77\%$
- F1Score: $= 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall} = 2 \cdot \frac{0.62 \cdot 0.35}{0.62+0.35} = 0,4488$

Ejercicio

Interpreta la siguiente matriz de confusión. Clasificar canciones según el subgénero de música electrónica al que pertenecen



5.1.3 Técnicas de Aprendizaje automático supervisado

A continuación vamos a describir algunos de los principales algoritmos clásicos de aprendizaje supervisado. Dejaremos las redes de neuronas para un tema posterior donde le dedicaremos mayor atención.

5.1.3.1 Linear regression model

- Usa la ecuación:

$$Y = w * x + b => f(w, b) = w * x + b$$

- Cómo se calcula w y b?
- Hay diferentes técnicas
  - Minimos cuadrados
  - Descenso de gradiente

Mean square error (MSE)

$$MSE(Y, Y') = \frac{1}{m} \cdot \sum_{i=1}^{m} (y_i - y'_i)^2$$

- Distancia entre y'_i (estimada => $f_{w,b}(x)$ en regresión lineal) y el valor real y_i al cuadrado para mantener el valor siempre positivo.
- Se suman todas las desviaciones y se calcual la media.
- En la práctica se suele dividir entre dos porque nos ayuda posterioremente

$$MSE(Y, Y') = \frac{1}{2m} \cdot \sum_{i=1}^{m} (y_i - y'_i)^2$$

La meta de la regresión lineal => Gradieent Descent

Minimizar la función de coste.

- Inicializamos los pesos w y b ocn valores aleatorios.
- Minimizamos $J(w, b)$
- Sigue cambiando w,b hasta que nos establezcamos cerca del mínimo

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

- α es la cantidad en que desplazamos w y b en cada iteración (tasa de aprendizaje / learning rate)

Gradient Descent algorithm

Repeat until convergence.

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (y_i\prime - y_i) * x_i$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (y_i\prime - y_i)$$

5.1.3.2 Logistic regression

Se usa para clasificación, es similar al anterior pero cambia la función por una función logistica:

Logistic function: $g(z) = \frac{1}{1+e^{-z}}\$$
La función de coste:

$$J(\vec{w}, b) = \frac{1}{2m} \cdot \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}_i) - y_i)^2$$

Y el descenso de gradiente:

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} y'_i - y_i x_{j,i}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} y'_i - y_i$$

donde $y' = f_{\vec{w},b}(\vec{x}_i) = \frac{1}{1+e^{-(\vec{w}\vec{x}+b)}}$

Para clasificación binaria, pero peude extenderse facilmente a multiclass aplicando recursivamente el algoritmo sobre 1 clase vs el resto e ir iterando sobre la clase objetivo.

Linear y Logistic regresión en SKLearn

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
X, y = load_iris(return_X_y=True)
clf = LogisticRegression(random_state=0).fit(X, y)
clf.predict(X[:2, :])
>> array([0, 0])
clf.predict_proba(X[:2, :])
>> array([[9.8...e-01, 1.8...e-02, 1.4...e-08],
       [9.7...e-01, 2.8...e-02, ...e-08]])
clf.score(X, y)
>> 0.97..
```

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
# y = 1 * x_0 + 2 * x_1 + 3
y = np.dot(X, np.array([1, 2])) + 3
reg = LinearRegression().fit(X, y)
reg.score(X, y)
>> 1.0
reg.coef_
>> array([1., 2.])
reg.intercept_
>> 3.0...
reg.predict(np.array([[3, 5]]))
>> array([16.])
```

5.1.3.3 K-Nearest Neighbor o k vecinos más cercanos

También conocido como **KNN** por sus siglas en inglés.

El algoritmo no intenta extraer una característica de los datos haciendo un proceso de entrenamiento exhaustivo con ellos. Simplemente los almacena para recuperarlos en el futuro.

Cuando se presenta un nuevo caso, éste es comparado con la base de casos y se extrae aquel más similar (o los K más similares) para comprobar cual era la clase a la que pertenecía. Se asume que si el caso es similar, la solución será similar.

Se necesita definir una función de similitud entre dos ejemplos. La calidad de esta función de similitud es crítica para éste algoritmo. Si la función de similitud no es capaz de medir correctamente la similitud que existe entre dos casos dados, la asunción de que a similares casos se espera que la instancia sea de la misma clase pierde vigencia y KNN deja de clasificar correctamente.

La solución a seleccionar depende del K elegido. Si K = 1 será la instancia recuperada más similar. Si K > 1 habrá que tomar una decisión de qué clase elegimos. Normalmente el criterio más lógico es **la clase mayoritaria**. Si la salida es un número real, podríamos hacer la media.

Hay multitud de **medidas de distancia** que podemos usar:

- **Distancia Euclídea**: Para cada atributo de X e Y

$$D - Euclidea(X, Y) = \sqrt[2]{\sum_{i=1}^{N} (x_i - y_i)^2}$$

- **Distancia de Manhattan**: Para cada atributo de X e Y

$$D - Man(X, Y) = \sum_{i=1}^{N} |x_i - y_i|$$

- **Distancia de Minkowski**: generalización de la euclídea o la de manhattan donde el exponente y la raíz pueden ser cualquier número.

$$D - Minkowski(X, Y) = \sqrt[p]{\sum_{i=1}^{N} (x_i - y_i)^p}$$

- **Distancia de edición**: el número de cambios que hay que hacer para convertir una instancia en otra. Generalización de Hamming.
  - casa → cala (sustitución de 's' por 'l') = 1
  - casa → calle (sustitución de s por L, sustitución a por e e inserción de l) = 3

- **Distancia de Mahalanobis**: la similitud entre dos variables aleatorias multidimensionales. Se diferencia de la distancia euclídea en que tiene en cuenta la correlación entre las variables aleatorias.

Formalmente, la distancia de Mahalanobis entre dos variables aleatorias con la misma distribución de probabilidad $\vec{x}$ e $\vec{y}$ se calcula de la siguiente forma:

$$D - Mahalanobis(\vec{x}, \vec{y}) = \sqrt[2]{(x - \mu)^T * C^-1 * (x - \mu)}$$

Donde μ es la media de los datos y C-1 es la inversa de la matriz de covarianzas.

Ejemplo:

```
def mahalanobis(x=None, data=None, cov=None):
    """Computa la covarianza entre cada fila de los datos
    x    : Vector o matriz de datos con p columnas
    data : ndarray el total de datos de la distribución
    cov  : matriz de covarianzas (p x p) de la distribución. Si no se invoca se establece como none None
    """
    x_minus_mu = x - np.mean(data) #Cálculo de la media
    if not cov: #Calculamos la covarianza i
        cov = np.cov(data.values.T)
    inv_covmat = sp.linalg.inv(cov) # inversión de la matriz de covarianza
    left_term = np.dot(x_minus_mu, inv_covmat) # producto escalar entre x menos la media y la inversa de la matriz de covarianza
    mahal = np.dot(left_term, x_minus_mu.T)
    # producto escalar entre la traspuesta de x menos la media y el cálculo anterior
    return mahal.diagonal() # devolvemos la diagonal de la matriz
```

Nótese que en el ejemplo no calculamos la raiz cuadrada, por lo que en realidad estamos calculando el cuadrado de la distancía de mahalanobis

otra aproximación es ponderar el peso que tiene cada uno de los atributos en el cálculo de la distancia. Esto implica **introducir información del dominio** ya que debemos saber o intuir que ciertas variables aportan mas información a la hora de decidir cuales coger.

$$D - Euclidea - Ponderada(X, Y, P) = \sqrt[2]{\sum_{i=1}^{N} p_i (x_i - y_i)^2}$$

Donde P es el vector de pesos asociado a cada entrada.

En principio podemos ponderar cualquier distancia pero en unas tiene más sentido que en otras.

Ventajas de KNN

- No paramétrico (salvo que usemos distancias ponderadas). No hace suposiciones explícitas sobre la forma funcionar de los datos, evitando los peligros de la distribución subyacente de los datos.
- Algoritmo simple tanto de explicar como de interpretar.
- Alta precisión (relativa). Es bastante alta, aunque no superior a otros modelos más sofisticados. Pero a pesar de su aparente simpleza, si se elige correctamente la distancia puede ofrecer resultados bastante buenos
- El proceso de entrenamiento es inmediato

Inconvenientes del KNN

- Es muy sensible a los atributos irrelevantes. Hacer una buena selección de atributos relevantes es fundamental.
- Es sensible al ruido, ya que, si un ejemplo es un mal ejemplo de entrenamiento y es el seleccionado como el más similar, daremos una solución errónea. Esto se puede mitigar haciendo que K sea grande ya que reduce el ruido
- La ejecución es lenta si hay muchos datos de entrenamiento, ya que tiene que procesar todos los datos. Existen métodos para optimizarlo usando partición espacial pero aún asi es costoso
- Es caro en memoria ya que ocupa mucha memoria si hay muchos casos (Y tiene difícil solución, salvo limitar la memoria de trabajo)

Ejemplo de KNN en SKLearn

```
print(len(iris.data))
X_train = iris.data[:-20]
y_train = iris.target[:-20]
X_test = iris.data[-20:]
y_test= iris.target[-20:]

from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors = 1)

model.fit(X_train, y_train)

prediction = model.predict(X_test)
print("Predicción")
print(prediction)
print("Test")
print(y_test)

model.score(X_test, y_test)
```

5.1.3.4 Árboles de decisión

Son muy útiles para visualizar las diversas opciones de las que se dispone para resolver un problema o modelar un comportamiento. Se pueden convertir en reglas.

Se pueden aplicar métodos de inducción hacia atrás para descubrir el razonamiento que encierra la lógica del sistema.

Un **árbol de decisión** está compuesto de dos tipos de nodo, un **nodo condicional** y una **clase**. Los nodos condicionales son los nodos internos del árbol y la clase, los nodos hoja o terminales.

Cada condición (nodo condicional) es una pregunta para el sistema acerca de una variable y tiene dos posibilidades, que sea cierta o falsa. En función de eso seguirá un camino y otro. Los nodos finales son nodos hoja y nos dice la clase.

Fuente (https://es.wikipedia.org/wiki/Árbol_de_decisión#/media/Archivo:Arbol_decision.jpg)

Algunas de las implementaciones más famosas de estos algoritmos son el **ID3, J48 o C4.5 (clasificación) y M5 (regresión)**

Este algoritmo utiliza la **entropía** (medida de incertidumbre o de desorden) para ayudar a decidir qué atributo debe ser el siguiente en ser evaluado en el árbol. Es decir, el atributo seleccionado es aquel que deja la información más ordenada o, mejor clasificada.

$$Entropia(s) = \sum_{n=1}^{c} -p_i * Log_2 p_i$$

Donde c son los posibles valores de clasificación, S es el conjunto de todos los ejemplos y p_i es la proporción de ejemplos de S que están en la clase i

Y la ganancia:

$$Ganancia(S, A) = Entropia(S) - \sum_{v \in V(A)}^{|A|} \frac{|sv|}{|s|} Entropia(S_v)$$

Donde V(A) es el conjunto de todos los valores posibles para el atributo A y Sv es un subconjunto de S para el cual el atributo A tiene el valor v

Como selecciona el atributo más prometedor de entre todos, podemos concluir que realiza una **búsqueda voraz** entre los mejores atributos. Después aplica **Divide y vencerás** recursivamente con el problema.

Algoritmo

```
ID3(E,A,N):N
E: ejemplos, A: atributos, N: nodoRaíz
-----------------------------------
Si vacio(A) o Todos lso ejemplos de E pertenecen a una clase
  N.Clase = ClaseMayoritaria(E)
si no
  a = MejorAtributo(A) => aplicando entropía
  A = A - a => quitamos a
  Para cada valor v del atributo a de todas las instancias
    newNode = CrearNodo(a,v)
    N.AddHijos(newNodo)
    E = E - aquellos cuyo valor de de a sea v
    ID3(E,A,newNode)
return N
```

Ejemplo

Sitio de acceso A1	Cantidad gastada A2	Vivienda A3	Última compra A4	Clase
1	0	2	Libro	Bueno
1	0	1	Disco	Malo
1	2	0	Libro	Bueno
0	2	1	Libro	Bueno
1	1	1	Libro	Malo
2	2	1	Libro	Malo

Elegimos el mejor atributo:
$A1 = \frac{1}{6}.I_{10} + \frac{4}{6}.I_{11} + \frac{1}{6}.I_{12}$ Donde I_{1i} son los posibles valores del atributo 1 = {0,1,2}
Ahora calculamos I_{10}, I_{11} e I_{12} usando la Entropía:
I_{10} tenemos para la clase 0 (Bueno) y valor 0 un total de 1 ejemplos. Para la clase 1 (Malo) y valor 0 tenemos 0 ejemplos. El total de elementos con el valor 0 es 1, de ahi podemos deducir lo siguiente:

$I_{10} = -\frac{1}{1}log_2\frac{1}{1} - \frac{0}{1}log_2\frac{0}{1} = -1 * 0 + 0 = 0$ y para el resto...
$I_{11} = -\frac{2}{4}log_2\frac{2}{4} - \frac{2}{4}log_2\frac{2}{4} = 1$ (hay 2 buenos y 2 malos)
$I_{12} = 0$ porque hay 0 buenos y 1 malo

Aplicando la formula tenemos que:

$A1 = \frac{1}{6}.0 + \frac{4}{6}.1 + \frac{1}{6}.0 = 0,66$

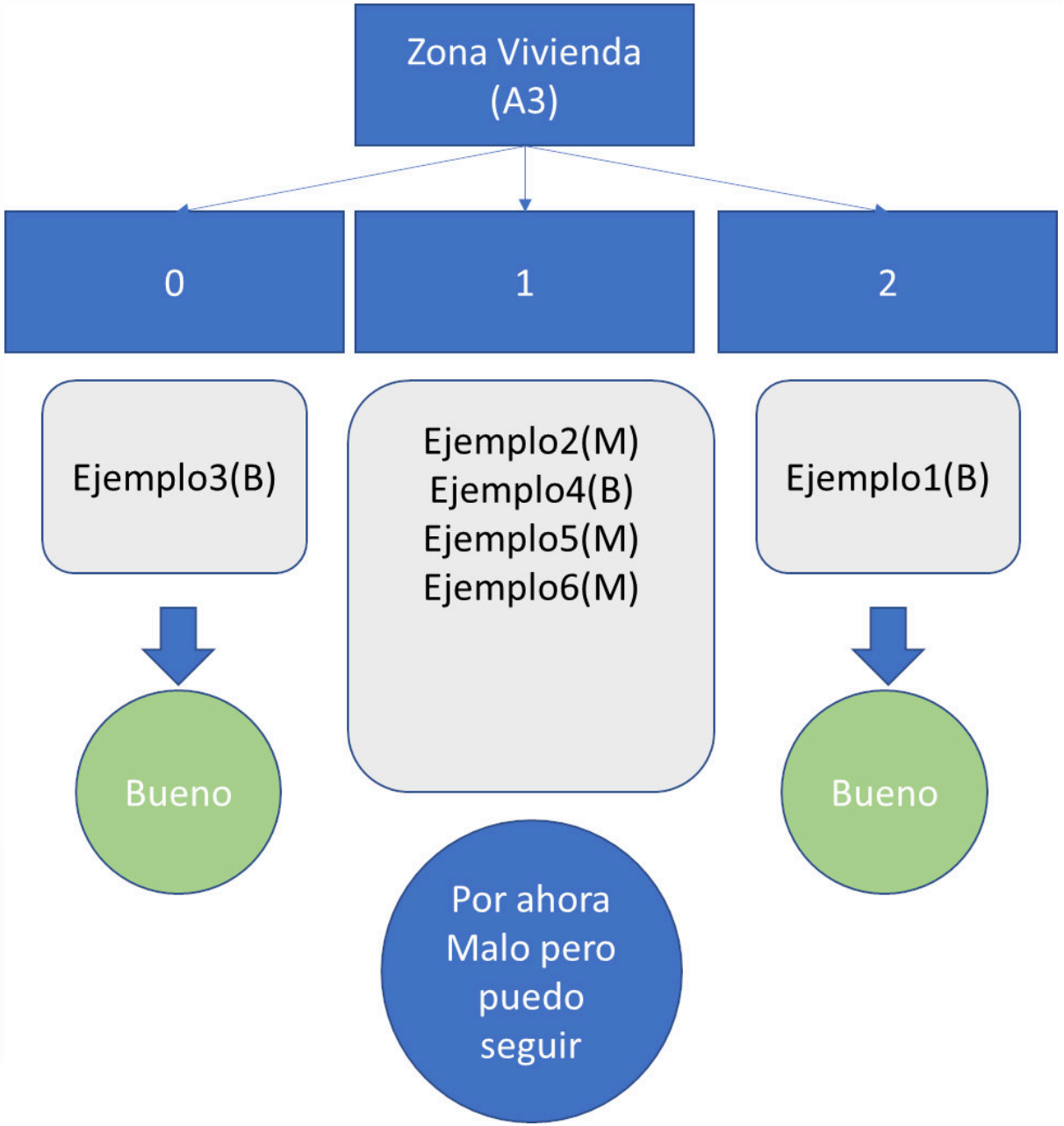
$A2 = 0,79$

$A3 = 0,54$

$A4 = 0,81$

El atributo seleccionado sería el de menor entropía (mayor orden) y por tanto seria el A3.

Esto ya nos deja el árbol con algunos terminales. Pero como puedo seguir podría seguir expandiendo y repitiendo la operación par el resto de atributos (A1,A2,A4)

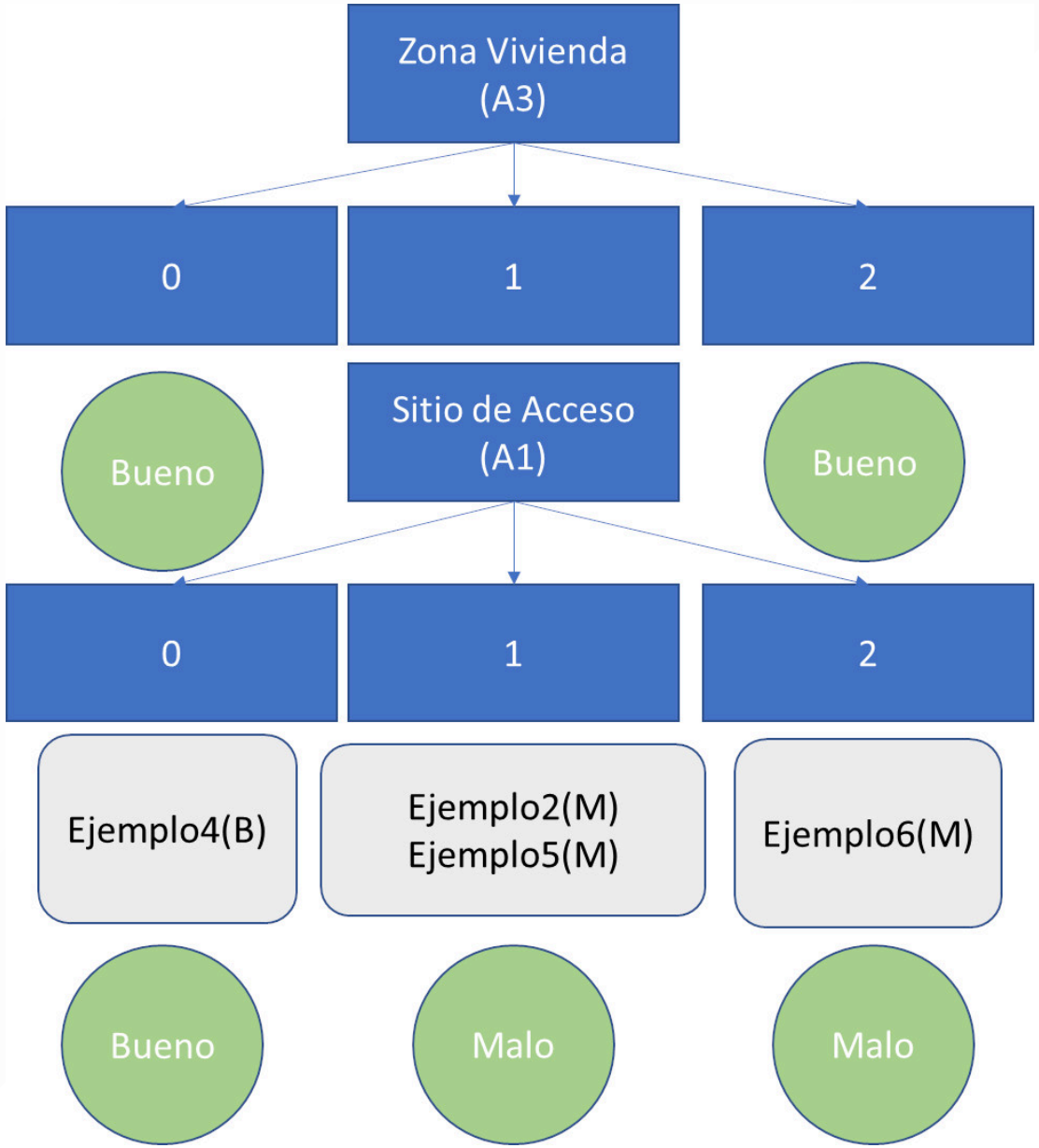

Zona Vivienda
(A3)
0
1
2
Ejemplo3(B)
Ejemplo2(M)
Ejemplo4(B)
Ejemplo5(M)
Ejemplo6(M)
Ejemplo1(B)
Bueno
Bueno
Por ahora
Malo pero
puedo
seguir

El siguiente atributo sería A1 sitio de acceso

$A1 = 0$
$A2 = 0,5$
$A4 = 0,23$

Que nos dejaría el árbol totalmente ordenado.

Nótese lo comentado antes, el DT hace una búsqueda voraz, jamas se replantea si habiendo puesto otro nodo inicialmente la configuración hubiera sido mejor (que en este caso no, pero en otros podría serlo.


Zona Vivienda
(A3)
0
1
2
Bueno
Sitio de Acceso
(A1)
Bueno
0
1
2
Ejemplo4(B)
Ejemplo2(M)
Ejemplo5(M)
Ejemplo6(M)
Bueno
Malo
Malo

Representación gráfica del algoritmo.

Los árboles de decisión dividen el espacio de soluciones mediante hiperplanos fijando una de las variables a un valor frontera.

Ventajas

- El entrenamiento es muy rápido
- Es fácil de interpretar los resultados por un humano, es un algoritmo de caja blanca.
- Para algunos problemas consigue una buena precisión.
- Se pueden convertir fácilmente en reglas.
- No requiere una preparación de los datos demasiado exigente.
- Puede trabajar con variables cualitativas y cuantitativas.

Desventajas

- Es muy dependiente al ruido de la entrada
- Los árbol de decisión tienden al sobre-entrenamiento
- No se puede garantizar que el árbol generado sea el óptimo
- Hay conceptos que no son fácilmente aprendibles pues los árboles de decisión ya que las particiones del espacio de soluciones que puede hacer son aquellas que son representables mediante una sucesión de hiperplanos. Si no hay una aproximación lineal al problema, puede que den un modelo poco efectivo.
- Se recomienda balancear el conjunto de datos antes de entrenar.

Ejemplo de DT en SKLearn

```
from sklearn.tree import DecisionTreeClassifier

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
iris = load_iris()
decision_tree = DecisionTreeClassifier(random_state=0, max_depth=2)
X_train = iris.data[:-20]
y_train = iris.target[:-20]
X_test = iris.data[-20:]
y_test= iris.target[-20:]

decision_tree = decision_tree.fit(X_train, y_train)
r = export_text(decision_tree, feature_names=iris['feature_names'])
print(r)
decision_tree.score(X_test, y_test)
```

Ejemplo de DT en SKLearn

```
|--- petal width (cm) <= 0.80
|    |--- class: 0
|--- petal width (cm) >  0.80
|    |--- petal width (cm) <= 1.65
|    |    |--- class: 1
|    |--- petal width (cm) >  1.65
|    |    |--- class: 2
```

```
0.9
```

Versiones extendidas: Random Forest

Este método ejecuta diferentes árboles de decisión y se realiza un proceso de votación para elegir cuál es la predicción final. Los RF son métodos de conjunto (ensamble methods) por este motivo. El número de árboles que genera es un parámetro del sistema:

- Dividimos nuestra serie de datos en varios subconjuntos compuestos aleatoriamente de muestras
- Se entrena un modelo en cada subconjunto
- Para cada nodo, elegir aleatoriamente m variables en las cuales basar la decisión. m debe ser < que el número total de variables
- Se combinan todos los resultados de los modelos (por votación)

Estos métodos obtienen mejores resultados en general que los árboles de decisión convencionales, aunque dificultan la comprensión del modelo generado.

Ventajas:

- Generalmente genera resultados muy buenos.
- Fácil de calcular
- Dar estimaciones de qué variables son importantes para clasificar

Desventajas:

- Sobreajusta si hay mucho ruido
- Es difícil de interpretar.

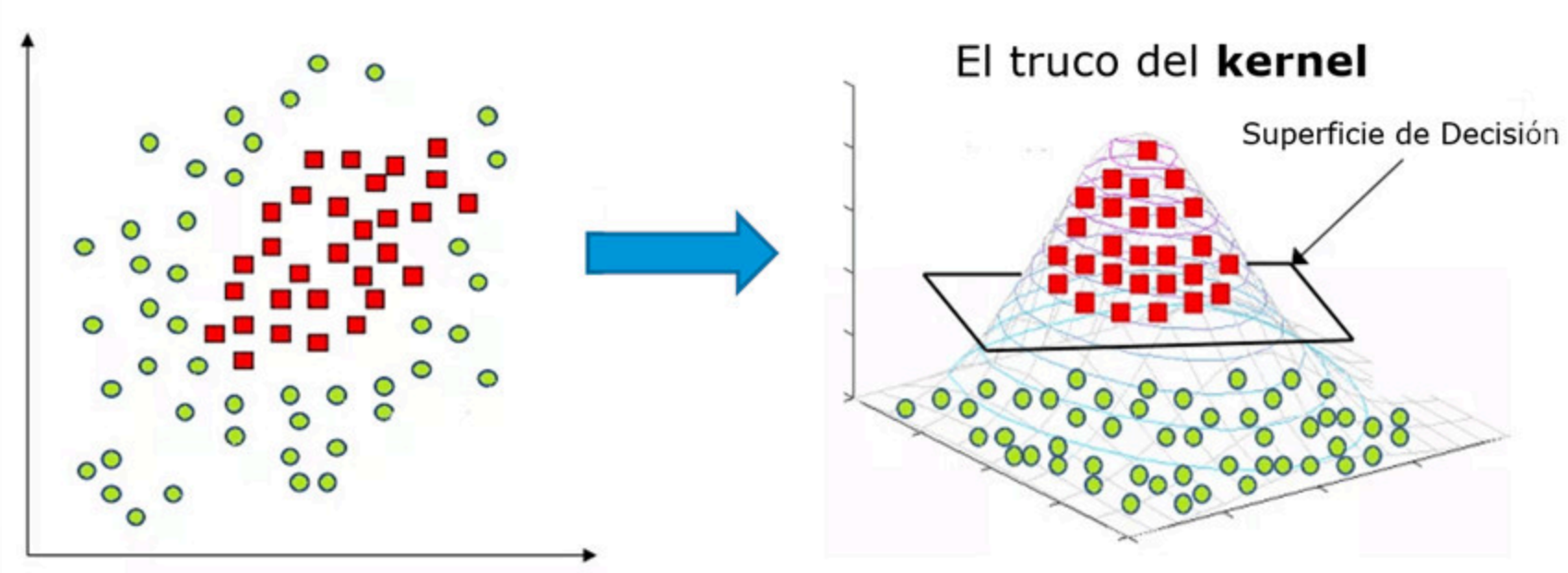
5.1.3.4 Support Vector Machine

Son un conjunto de algoritmos de aprendizaje supervisado desarrollados por **Vladimir Vapnik** y su equipo en los laboratorios AT&T.

Estos métodos están propiamente relacionados con problemas de clasificación y regresión.

Los vectores de soporte son los puntos que definen el margen máximo de separación del hiperplano que separa las clases. Se llaman vectores porque estos puntos tienen tantos elementos como dimensiones tenga nuestro espacio de entrada.

Hay veces en las que no hay forma de encontrar un hiperplano que permita separar dos clases. En estos casos, decimos que las clases no son linealmente separables. Para resolver este problema podemos usar un **kernel**. El truco del kernel consiste en inventar una dimensión nueva en la que podamos encontrar un hiperplano para separar las clases.



Ventajas

- Eficaz en espacios de grandes dimensiones.
- Todavía eficaz en casos donde el número de dimensiones es mayor que el número de muestras.
- Utiliza un subconjunto de puntos de entrenamiento en la función de decisión (llamada vectores de soporte), por lo que también es eficiente en memoria.
- Versátil: se pueden especificar diferentes funciones del núcleo para la función de decisión. Se proporcionan kernels comunes, pero también es posible especificar kernels personalizados.

Desventajas

- Si el número de características es mucho mayor que el número de muestras evite el exceso de ajuste al elegir las funciones del Kernel y el término de regularización es crucial.
- Los SVMs no proporcionan directamente estimaciones de probabilidad, éstas se calculan utilizando una validación cruzada.

SVM en SKLearn

```
from sklearn.svm import SVC
classifier = SVC(kernel = 'linear', random_state = 0)
#Fit the model for the data

classifier.fit(X_train, y_train)

#Make the prediction
y_pred = classifier.predict(X_test)
print(y_pred)
model.score(X_test, y_test)
```

Bibliografía

- Russell, S. J., & Norvig, P. (2004). Inteligencia Artificial: un enfoque moderno (No. 04; Q335, R8y 2004.).
- Millán, D. B., Boticario, J. G., & Viñuela, P. I. (2006). Aprendizaje automático. Sanz y Torres.
- Marsland, S. (2015). Machine learning: an algorithmic perspective. CRC press